

Algoritmos y Estructuras de Datos III

Trabajo Práctico N° 4	Unidad 3
Modalidad: Semi -Presencial	Estratégica Didáctica: Trabajo individual
Metodología de Desarrollo: Det. docente	Metodología de Corrección: Via Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	Fecha Entrega: A confirmar por el Docente.

PILAS Y COLAS.

MARCO TEÓRICO:

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1. Describir el concepto de Pila y sus Operaciones Básicas

Se denomina pila (stack) a una lista restringida en la que todas las inserciones y eliminaciones se realizan únicamente por un extremo llamado cima (top). Sólo puede accederse al elemento superior, no se pueden acceder directamente a posiciones intermedias o inferiores. Esta restricción le confiere su característica fundamental: el último elemento insertado es el primero en salir. Por eso se la llama estructura LIFO (Last In First Out). El material la ilustra con ejemplos cotidianos: una pila de libros o de platos, donde solo es posible añadir o quitar cosas desde arriba.

Las operaciones básicas son:

- **push(i, S)** — inserta el elemento i en la cima de la pila. Antes de ejecutarla se debe verificar que la pila no esté llena.
- **pop(S)** — elimina el elemento de la cima y lo devuelve. Antes de ejecutarla se debe verificar que la pila no esté vacía.
- **getTop(S)** — devuelve el elemento de la cima sin eliminarlo. Solo lee, no modifica la variable top. Es la diferencia clave con pop().
- **create(S)** — inicializa la pila en estado vacío. En la implementación con array, top se inicializa en -1.
- **isEmpty(S)** — devuelve true si $top == -1$, protege contra un pop() sobre pila vacía (underflow).
- **isFull(S)** — devuelve true si $top == MaxCapacity - 1$, protege contra un push() sobre pila llena (overflow).

2. Explicar el concepto de “StakOverFlow” y como evitarello?

El StackOverFlow (desbordamiento de pila) se produce cuando se intenta ejecutar `push()` sobre una pila que ya está llena. El material lo describe gráficamente como una pila de libros hasta el techo: ya no hay espacio para más. En informática, cuando el stack de llamadas se llena con exceso de subrutinas anidadas (como por ejemplo una recursión infinita), el sistema ya no puede crear más frames y se produce este error.

Con el fin de evitarlo, hay dos mecanismos:

- Comprobar `isFull()` antes de cada `push()`. Si la pila está llena, no se inserta y se da aviso del error.
- En recursión, evitar la recursión infinita garantizando siempre un caso base que detenga las llamadas antes de que el stack de llamadas se desborde.

3. Buscar describir las diferencias de Implementar Pilas en Clase so via STL?

implementación propia (clase)

Se define una clase `Stack` con un array interno, la variable `top` y métodos propios para ello (`push`, `pop`, `getTop`, `isEmpty`, `isFull`). El tamaño es estático y está fijo (se declara en tiempo de compilación). Tiene control total sobre la lógica interna y el manejo de errores. Puede ser extendida con templates (`template <class T>`) para hacerla genérica y soportar cualquier tipo de dato.

Implementación con STL (`<stack>`)

Usamos directamente `std::stack<T>` incluyendo `#include <stack>`. Es un adaptador de contenedor que internamente usa `deque` (por defecto), `vector` o `list`. No tenemos que implementar nada: las operaciones `push()`, `pop()`, `top()`, `empty()` y `size()` ya están disponibles y probadas, todas con complejidad $O(1)$. Es más concisa, portable y libre de errores de implementación.

La principal diferencia es que la propia clase permite controlar, mientras que la STL es más segura, genérica y lista para producción. La clase propia requiere gestionar manualmente el estado de overflow; la STL no tiene un límite de tamaño fijo (crece dinámicamente).

4. Que es una Cola y cuales son las operaciones Basicas?

Una cola es una lista ordenada donde las inserciones se hacen por un extremo llamado rear (parte trasera) y las eliminaciones por el otro extremo llamado front (parte delantera), a diferencia de la pila que usa un solo extremo. Es una estructura FIFO (First In First Out), es decir, el primer elemento en entrar es el primero en salir. Una fila de espera en la vida real: las personas se agregan por atrás y se atienden por delante.

Las operaciones básicas son:

- **create()** — crea una cola vacía. Front y Rear se inicializan en -1.
- **add(i, Q)** — agrega el elemento i al extremo Rear. Se verifica primero que la cola no esté llena.
- **delete(Q)** — elimina y devuelve el elemento del extremo Front. Se verifica que la cola no esté vacía.
- **getFront(Q)** — devuelve el elemento del frente sin eliminarlo (equivalente al `getTop()` de las pilas).
- **isEmpty(Q)** — devuelve true si $\text{Front} == \text{Rear}$.
- **isFull(Q)** — devuelve true si $\text{Rear} == \text{max} - 1$.

5. Describir: Cola circular, Multicola, Deque y cola prioritaria.

Cola circular (Cola circular)

La cola lineal con array presenta un problema: cuando Rear llega al final del array, señala `Queue_Full` incluso si existen posiciones vacías al inicio (liberadas por eliminaciones anteriores). La cola circular lo soluciona haciendo que Rear “envuelva” el inicio cuando llega al final. De esta manera el array se comporta como un anillo. La cola está realmente llena solamente cuando contiene n elementos. Dentro de las ventajas está que no desplaza datos, aprovecha toda la memoria disponible, y se emplea en aplicaciones prácticas como colas de impresión.

Multicola (Multiqueue)

Si se requieren varias colas simultáneas, un único array grande se puede dividir en n segmentos, asignando uno a cada cola. Cada cola i tiene su propio par `Front[i]` y `Rear[i]`. Cuando $\text{Front}[i] == \text{Rear}[i]$ significa que la cola i está vacía. Por ejemplo, un array `A[100]` con 5 colas de 20 posiciones cada una.

Cola de doble extremo (Deque)

El deque es una generalización de la pila y la cola que permite insertar y eliminar elementos por ambos extremos (frente y rear), pero no en posiciones intermedias. Sus cuatro operaciones son `EnqueueFront()`, `EnqueueRear()`, `DequeueFront()` y `DequeueRear()`. Hay dos variantes: la cola restringida de entrada (sólo inserta por una cara) y la cola restringida de salida (sólo suprime por una cara). La STL lo ofrece en `std::deque`, que tiene acceso $O(1)$ a ambos extremos.

Cola prioritaria

Es una colección de elementos con nivel de prioridad asignado a cada uno. Al eliminar, siempre se retira el elemento de mayor prioridad, independientemente del orden de inserción. Cuando dos elementos tengan la misma prioridad, se atiende el orden de llegada (FIFO entre iguales). Puede implementarse con arrays, listas enlazadas, árboles de búsqueda binaria o estructuras heap. Applications: Operating system scheduling, emergency rooms (patients by severity), event simulations.

6. Para Pilas y Colas mas subtipos, Dar ejemplos de Utilización de cada uno.

Pila (Stack):

- + Botón "Atrás" de un navegador web, cada URL visitada se apila y al volver se desapila.
- + Llamadas a funciones en el sistema operativo, cada llamada apila el estado (dirección de retorno, variables locales) y al retornar se desapila.
- + Verificación de paréntesis bien formados en un compilador.
- + Conversión de expresiones infix a postfix.
- + Algoritmos de backtracking (laberintos, juegos).

Cola (Queue):

- + Cola de impresión en red, los trabajos se encolan por orden de llegada y se imprimen en ese orden.
- + Sistema de turnos de atención al cliente
- + Gestión de procesos en un sistema operativo (scheduler).

Cola circular:

- + Buffer de datos en comunicaciones (transmisión de paquetes de red).
- + Cola de impresoras compartidas en una red.

Multicola:

- + Sistemas con múltiples categorías de atención simultánea (ej.: cajas de supermercado por tipo de producto).

Deque:

- + Historial de llamadas con navegación hacia adelante y atrás (como el botón Back/Forward de un navegador).
- + Implementación de algoritmos de búsqueda por niveles.

Cola prioritaria:

- + Sistema de emergencias hospitalarias, se atiende primero al paciente más grave.
- + Scheduler del sistema operativo, procesos con mayor prioridad se ejecutan antes.
- + Algoritmos de búsqueda como Dijkstra (menor costo primero).

MARCO PRÁCTICO

1. Un Negocio necesita gestionar la atención a sus cliente, el mismo recibe los clientes en un Box de Atencia que registra los clientes y los carga para su atención según su llegada. Nos piden desarrollar un sistema de "Turnos" que se muestren en una pantalla. La aplicación deberá registrar los clientes, mostrarlos en orden de llegada, llamarlos según ese orden por pantalla con una opción "próximo cliente – Box nro"
2. Ídem 1 pero Implementando Listas en STL
3. Una empresa necesita implementar un Historial de llamadas a sus cliente, la aplicación deberá ir cargando los números telefónicos en un historial y el usuario podrá ir recorriendo el histórico de llamadas (similar al botón Back del navegador). Implementar una app que realice esta función.